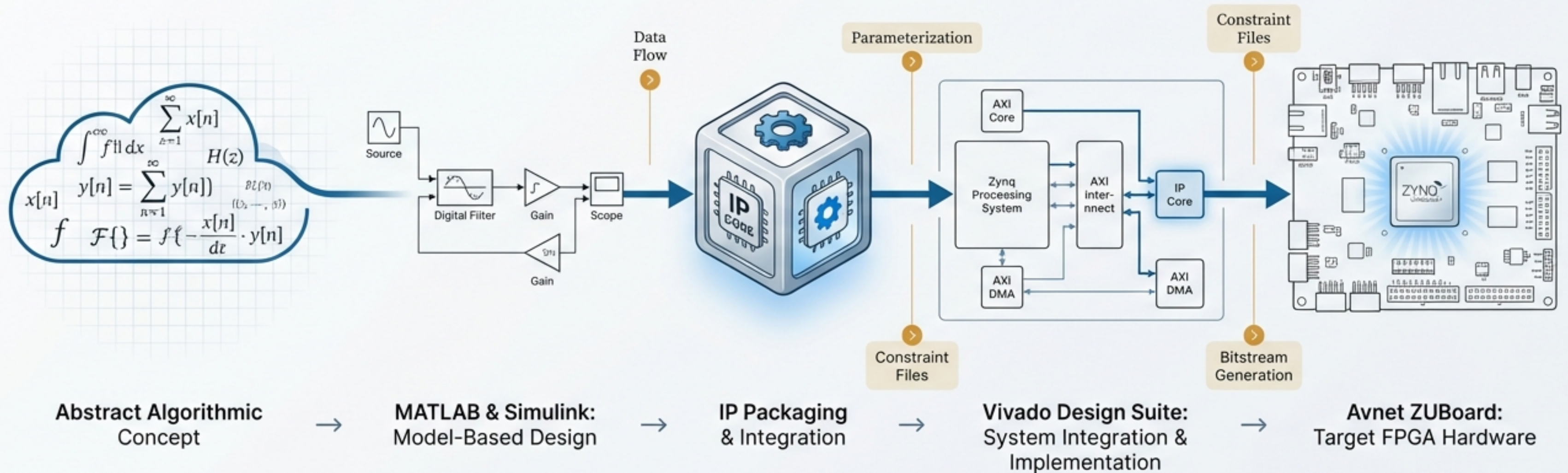
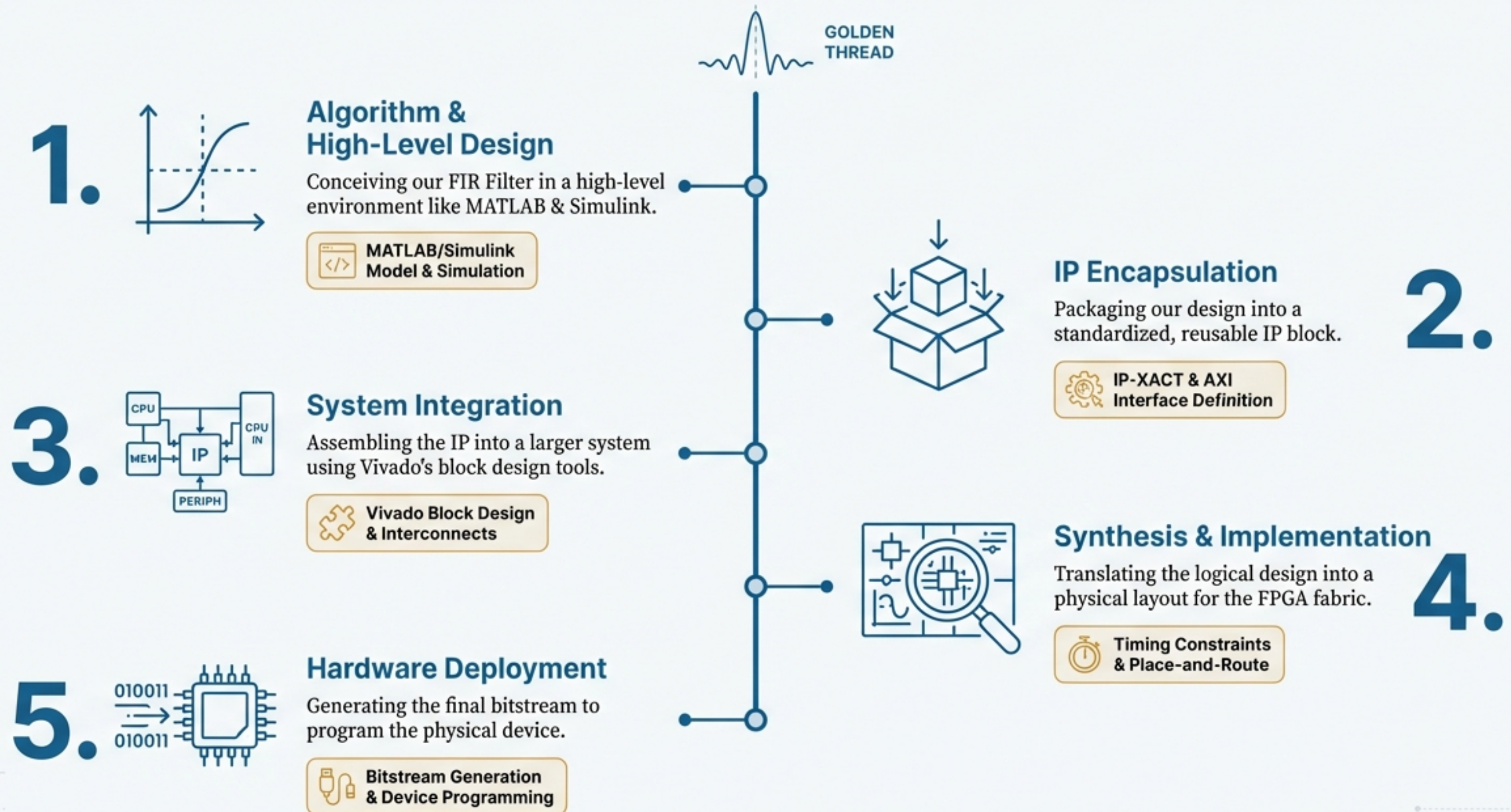


From Algorithm to Bitstream: A Modern FPGA Design Journey

Following a single design from a high-level model in MATLAB to
a fully implemented system on an FPGA using Vivado.



The Five Stages of Our Design's Journey



Stage 1: Starting with a 'Golden Reference' in MATLAB

 GOLDEN THREAD

The "Golden Reference"

The most effective workflow begins by using MATLAB to develop and verify a pure, frame-based algorithm. This model acts as the "golden reference" to validate against at every subsequent step.

The Transition to Hardware Thinking

The algorithm is then moved to Simulink to add implementation details like timing and data types. A key first step is converting the algorithm from processing entire data frames (e.g., an image) to processing a continuous stream of data (e.g., one pixel at a time). This shift is fundamental for efficient hardware design.

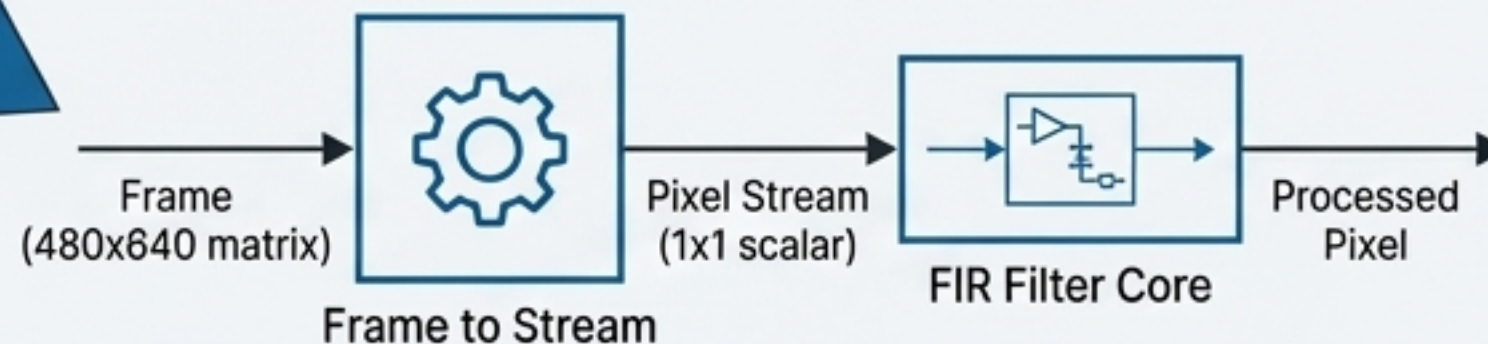
Frame-Based Algorithm

```
MATLAB - fir_filter.m
function output_frame = fir_filter(input_frame, coeffs)
    % Frame-based processing
    output_frame = conv2(input_frame, coeffs, 'same');
end
```



Process entire data frames at once.

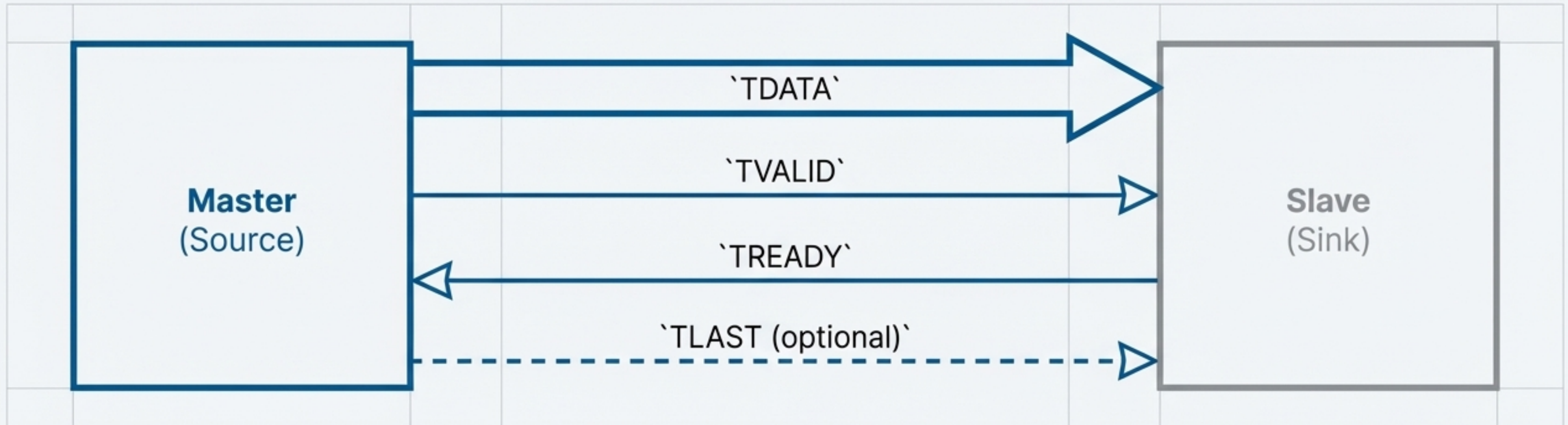
Stream-Based Implementation



Process a continuous stream of individual pixels.

The Language of Hardware Streams: Introducing AXI Stream

For hardware blocks to work together, they need a standard way to pass streaming data. AXI Stream is the most common interface for this. It's not just about data; it's a protocol with rules for flow control.



`TDATA`: The actual data payload.

`TVALID`: From the Master (source), indicating `TDATA` is valid.

`TREADY`: From the Slave (sink), indicating it is ready to accept data.

`TLAST`: An optional signal indicating the last piece of data in a packet.

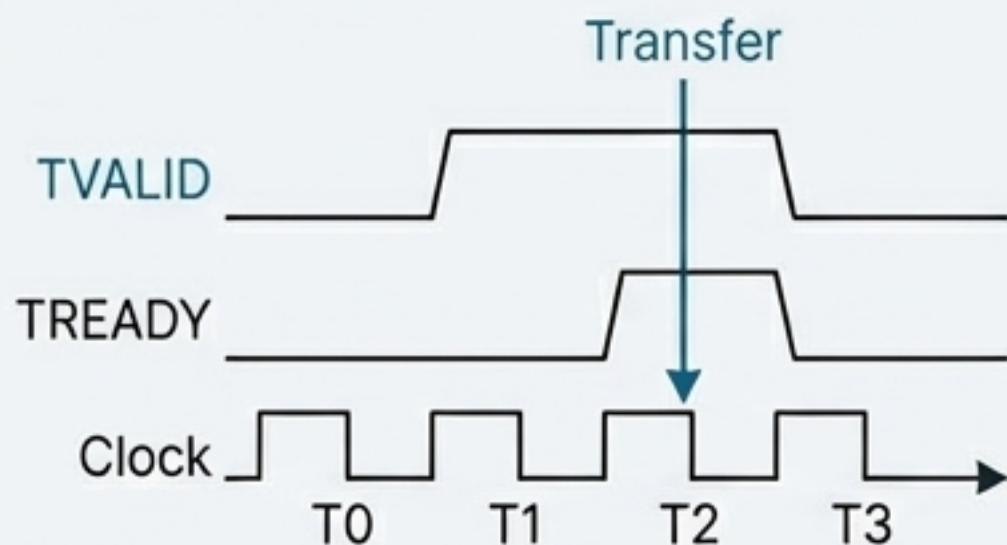
The Handshake is Everything: The One Rule You Must Remember

The Golden Rule

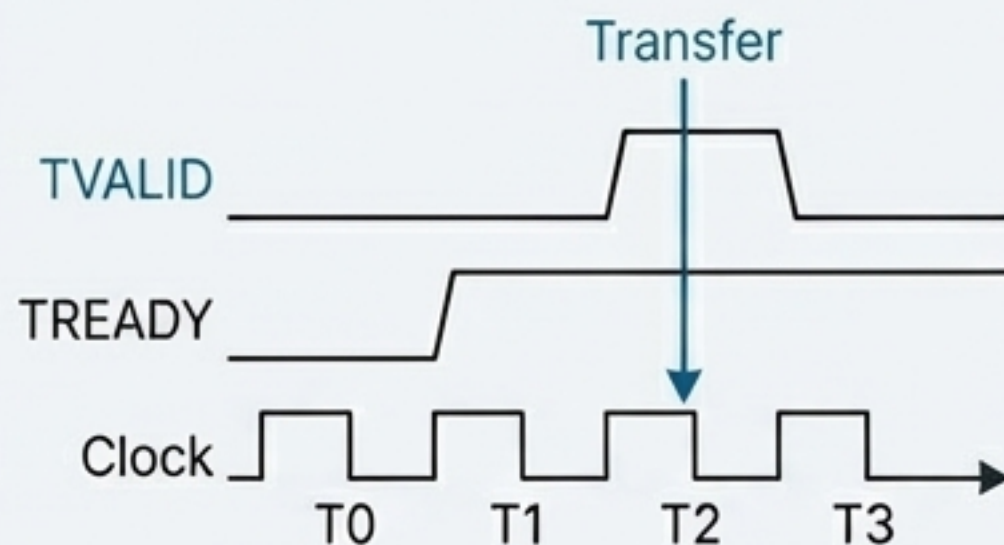
transfer = TVALID and TREADY

A data transfer *only* occurs on a clock cycle when 'TVALID' AND 'TREADY' are both high. This simple handshake is the core of the entire protocol.

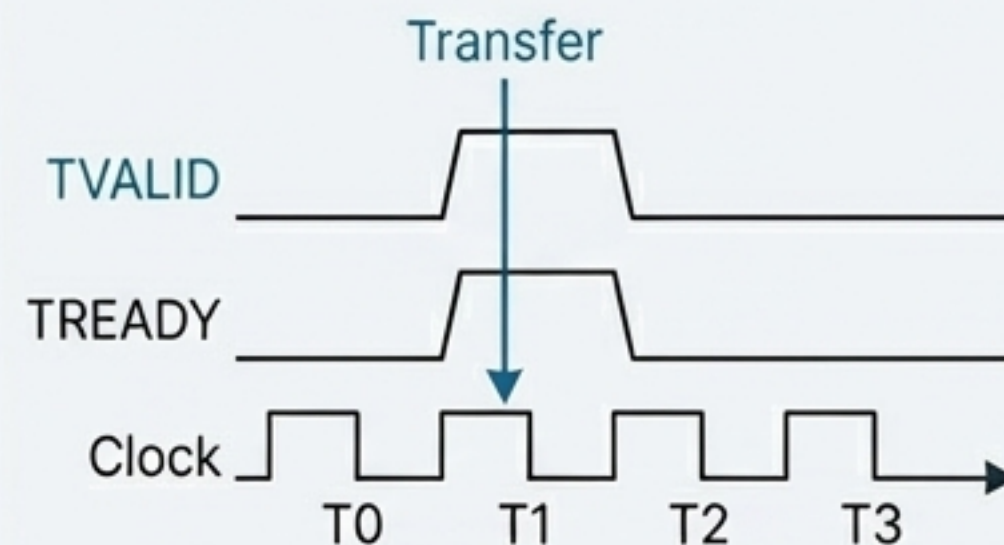
Scenario 1: Master Waits



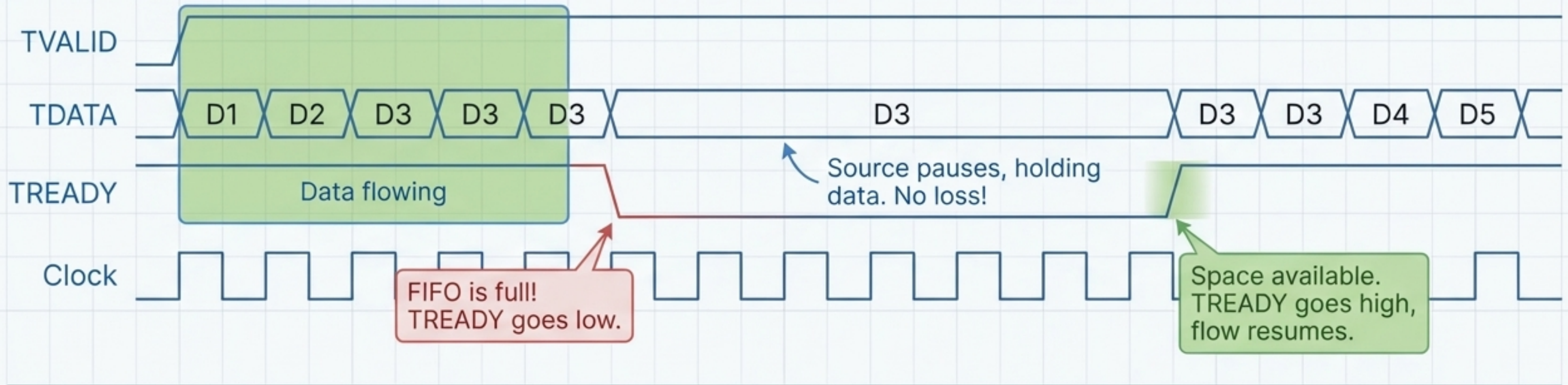
Scenario 2: Slave Waits



Scenario 3: Simultaneous



Visualizing Backpressure: How `TREADY` Prevents Data Loss



A fast source feeding a slow sink via a FIFO buffer.

1. Initially, the FIFO is not full, so it asserts `TREADY`, accepting data quickly.
2. When the FIFO fills up, it **de-asserts** `TREADY`, signaling to the source: “Hold on, I’m full.”
3. The source sees `TREADY` is low and, following the protocol, must hold its `TDATA` and keep `TVALID` high, pausing transmission without losing data.
4. As the slow component drains the FIFO, space becomes available, and the FIFO re-asserts `TREADY`, resuming the flow.

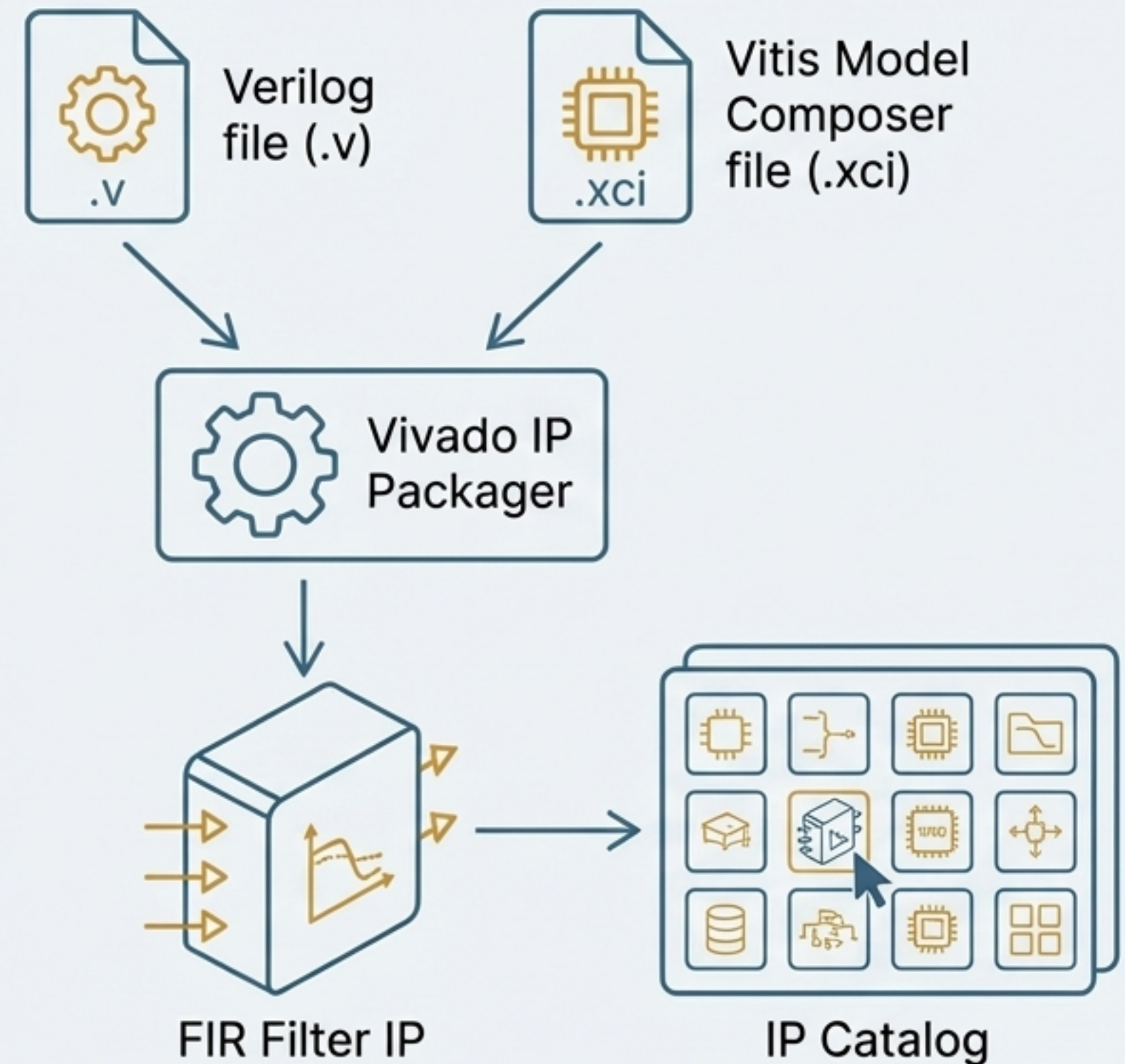
Stage 2: Encapsulating Your Logic as Reusable IP

Why Package IP?

Instead of a monolithic design, modern FPGA development relies on modular, reusable blocks. Packaging your HDL code or high-level model as an IP makes it easy to integrate, configure, and reuse in different projects.

The Vivado IP Packager Workflow

1. Provide your source files (Verilog/VHDL or generated netlists).
2. Vivado's tools help you define parameters (like filter coefficients or data widths).
3. You can customize the graphical user interface (GUI) for your IP.
4. The final step packages it into the IP Catalog, ready to be used in a block design.



The Power of Interfaces: Moving from Pins to Protocols

The Old Way: Pins

Manually connecting dozens of individual signals (`clk`, `reset`, `data\_in\_0`, `valid\_in`, etc.). This is tedious and error-prone.

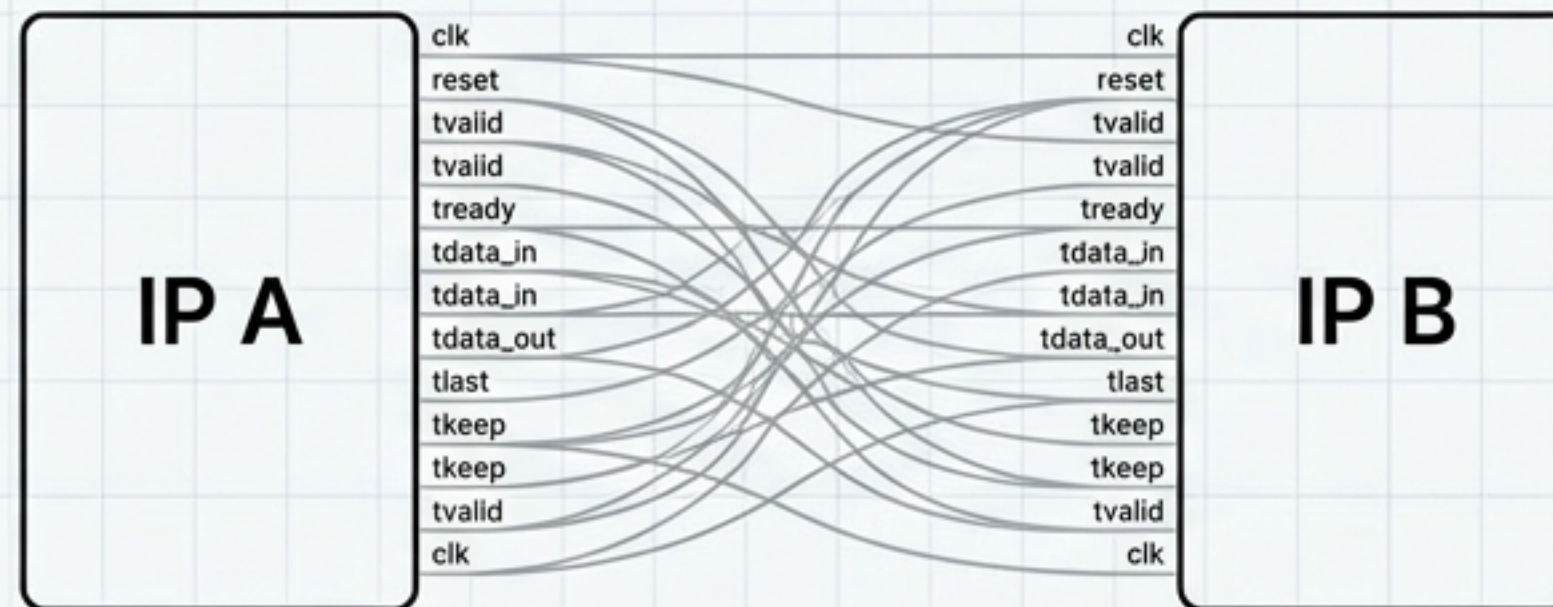
The Modern Way: Interfaces

During IP packaging, you group these individual pins and map them to a predefined interface standard, like `AXI4-Stream`.

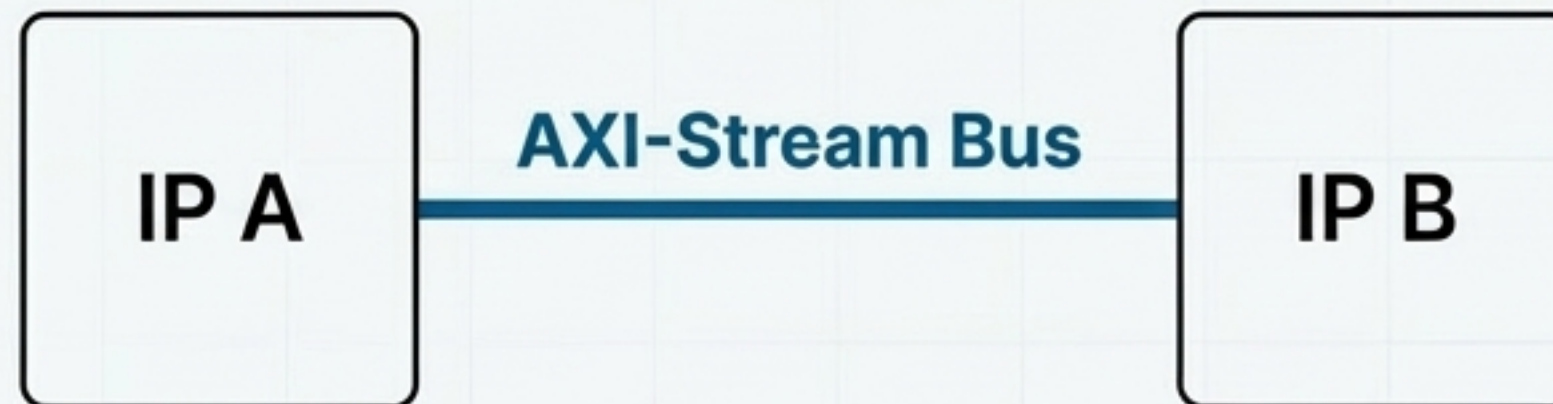
The Benefit: Simplicity

Vivado now understands this group of signals as a single, standardized bus. You can connect your IP to any other AXI Stream-compliant IP with a single line in the block design.

BEFORE



AFTER



Pro-Tips for Managing and Using Your IP



Tip 1: Version Control Your IP Correctly.

Store your IP in a dedicated directory *outside* your Vivado project folder. For version control (like Git), you only need to commit two files: the `.xci`` (core configuration) and `.xml`` (GUI definition). All other files can be regenerated.



Tip 2: Let Vivado Write Your Testbench.

Don't know how to use a complex IP? Right-click it in the IP Catalog and select "Open IP Example Design." Vivado will create an entirely new, fully functional project showing exactly how the IP is instantiated and tested.



Tip 3: Version Control Your Entire Project.

To save your entire project structure without all the generated files, use ``File > Project > Write Tcl``. This creates a script that can rebuild your project. Commit this Tcl script along with your HDL and IP source files.

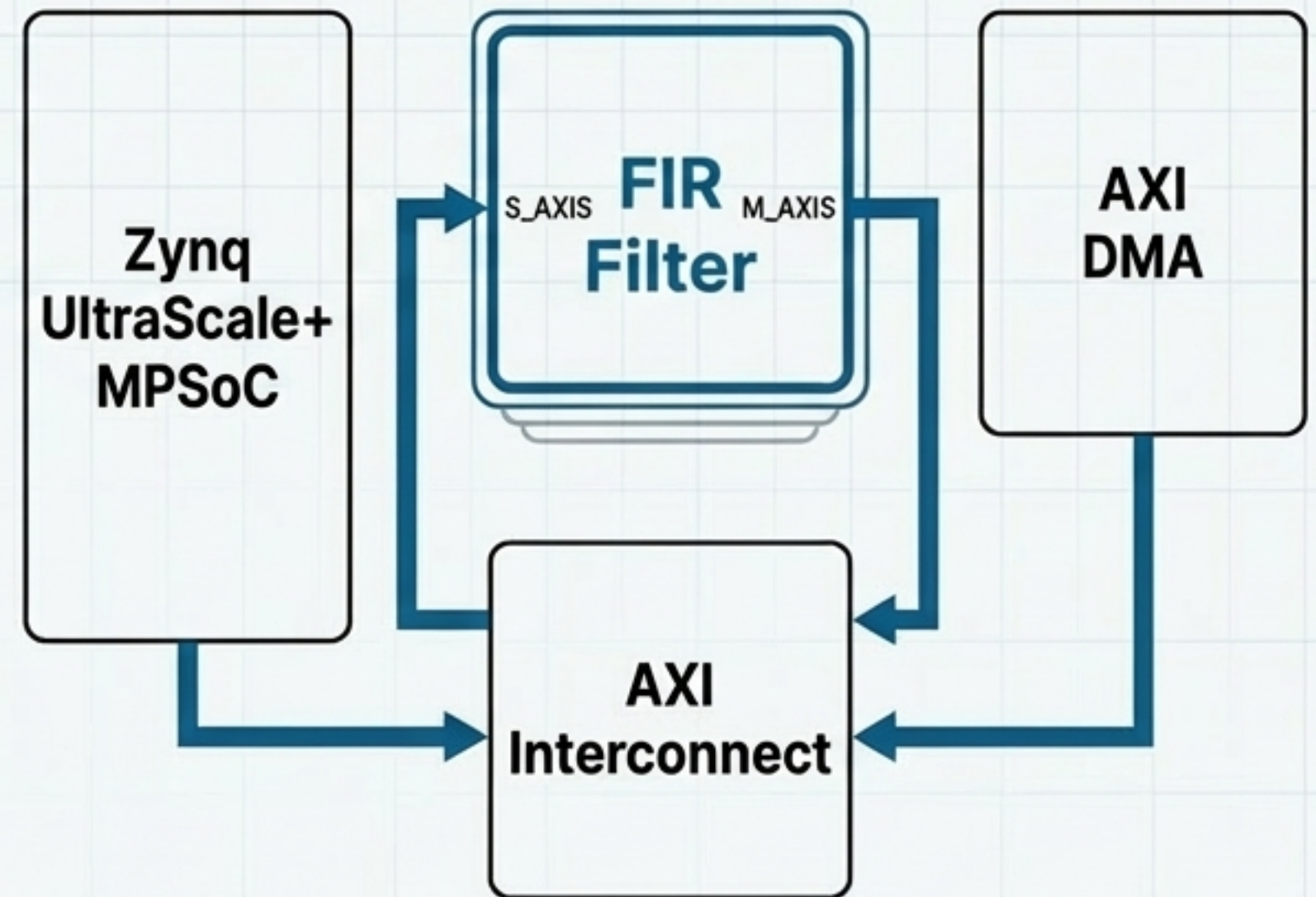
Stage 3: Assembling the System in a Block Design

The Process

Vivado's IP Integrator provides a graphical canvas to build your system. You simply drag-and-drop IP blocks from the IP Catalog—including the FIR Filter IP we just created.

Connecting the System

Because we used standard AXI interfaces, connecting our filter to other system components (like a DMA for memory access or a Zynq Processing System) is as simple as drawing lines between the bus interfaces. Vivado handles the complex pin-to-pin wiring automatically.



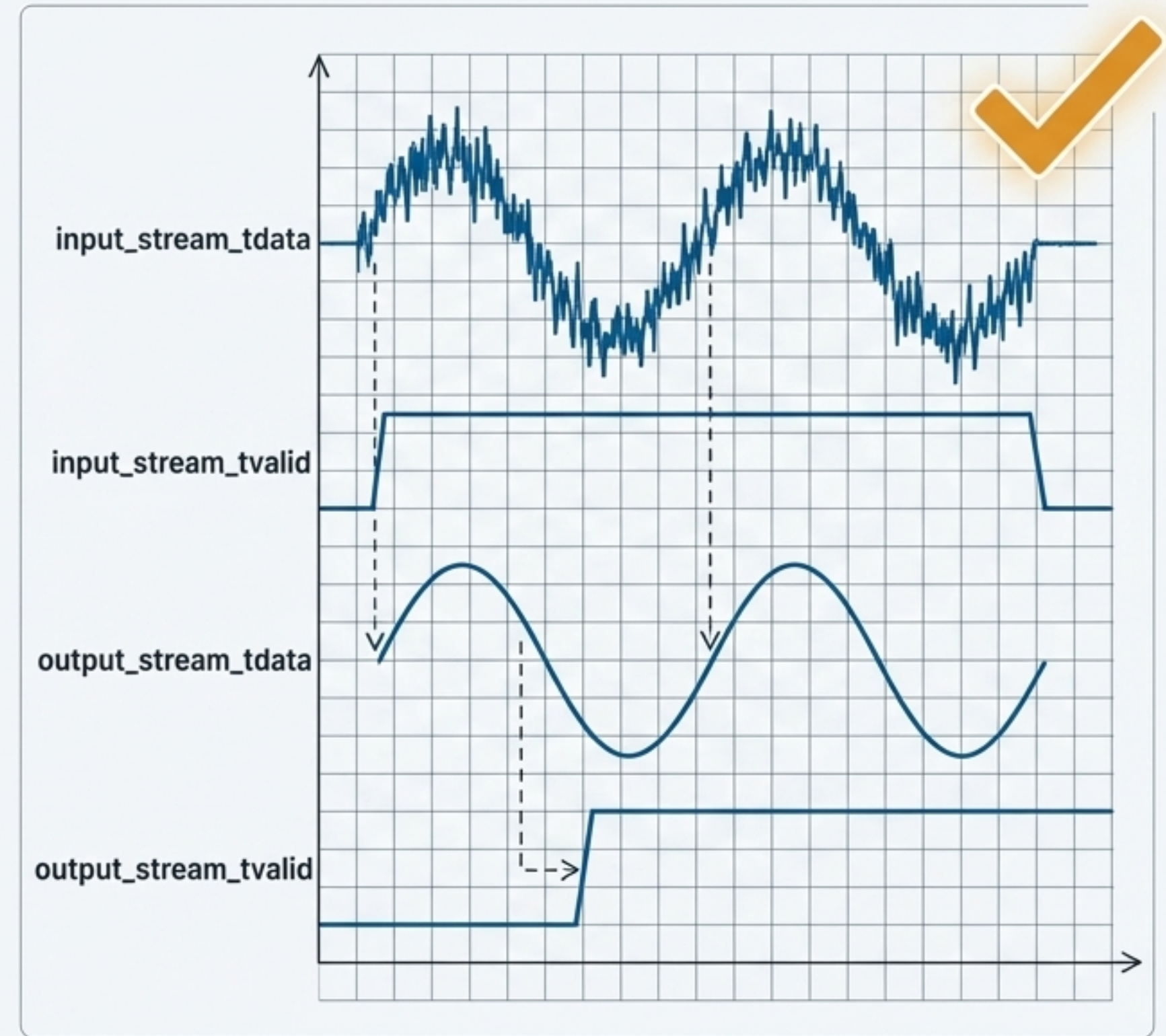
Verification Before Implementation: The Behavioral Simulation

Purpose

After connecting all the IP blocks, we run a "behavioral simulation" directly within Vivado. This simulation tests the functionality of the integrated system at a high level, confirming that data flows correctly and control signals behave as expected.

Why It's Critical

This step catches integration errors (e.g., incorrect AXI connections, data width mismatches) *before* the lengthy synthesis and implementation processes begin, saving hours or even days of development time.

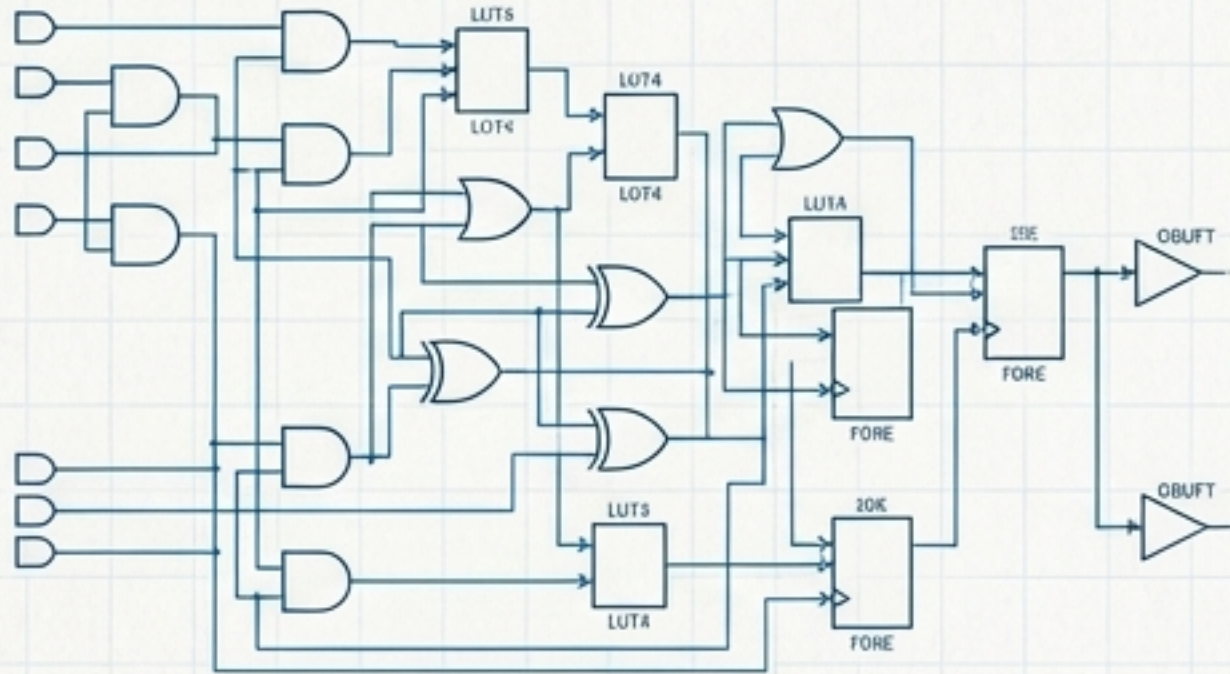


Stage 4: Translating Logical Design to Physical Layout

Step 1: Synthesis

An “estimation and a design planning step.” Vivado takes the block design and HDL sources and translates them into a netlist—a low-level description of logic gates (LUTs, Flip-Flops) and their connections.

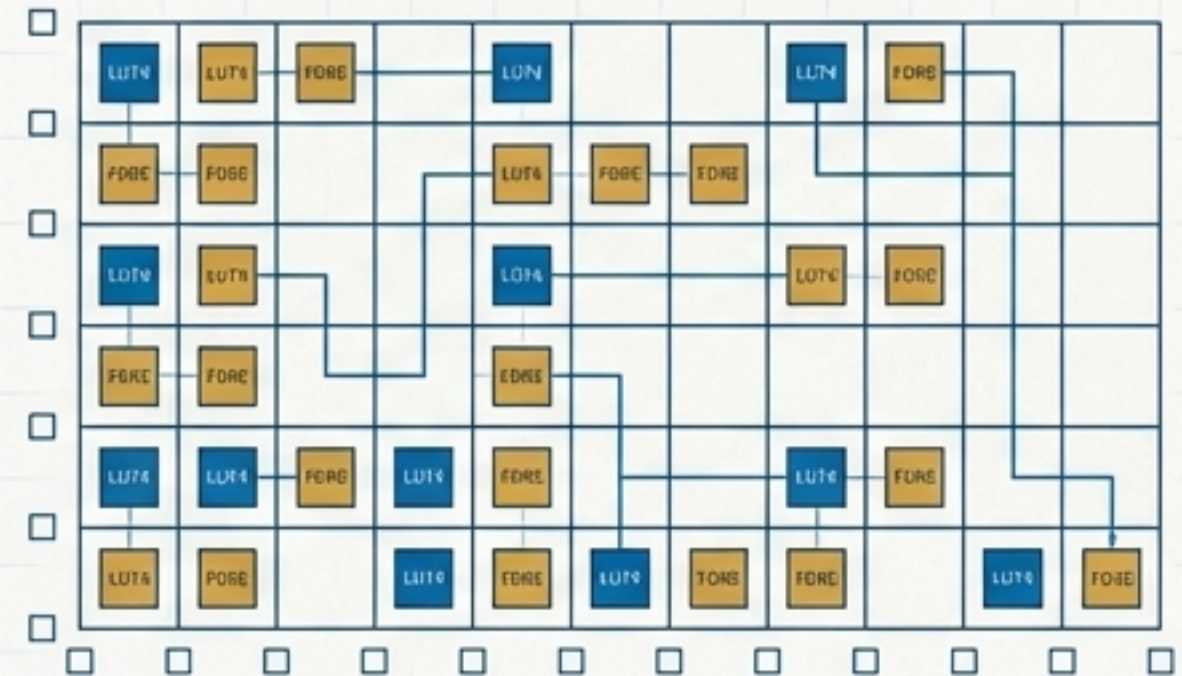
- Output: A logical representation of the design, optimized for the target FPGA architecture.



Step 2: Implementation

The “place and route” stage. The tools take the synthesized netlist and physically place each logic element onto a specific location on the FPGA die and route the wire connections between them.

- Output: A fully placed and routed design that maps directly to the physical silicon.



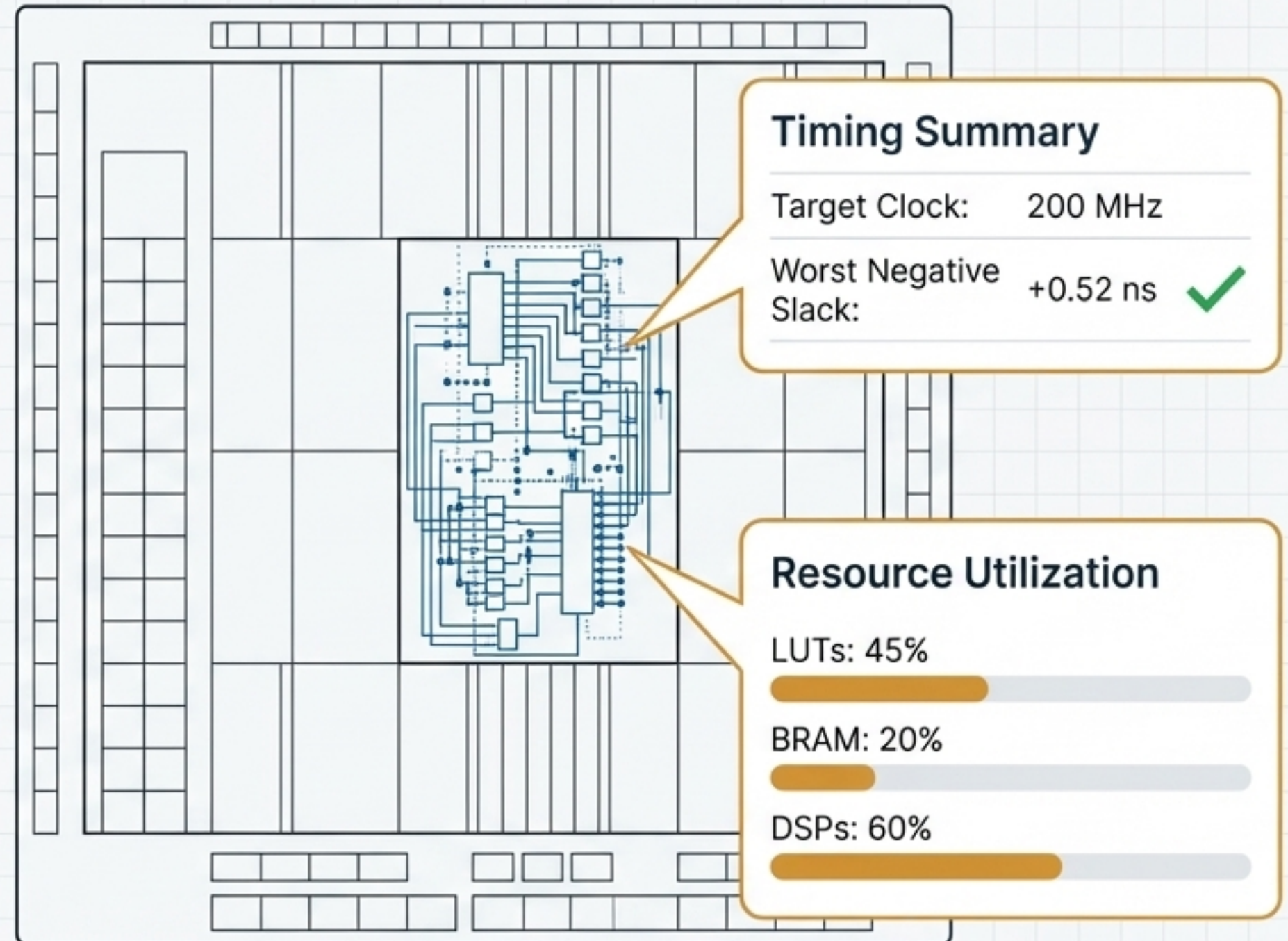
Reviewing the Results: Does the Design Meet Timing?

Post-Implementation Analysis

After place-and-route, Vivado provides detailed reports on the physical design. This is the first time we get real performance numbers.

Key Reports to Check

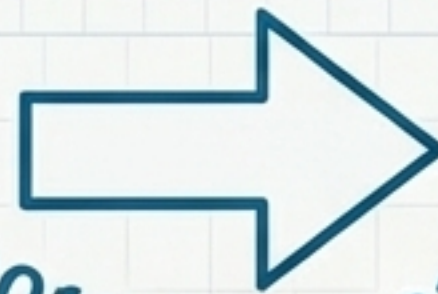
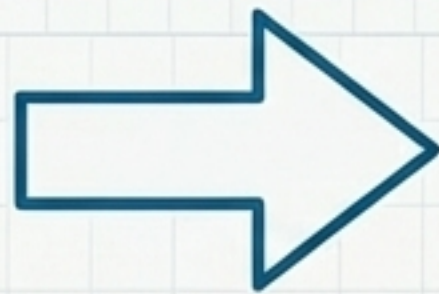
- **Utilization:** How much of the FPGA's resources (LUTs, BRAM, DSPs) did our design use? Are we close to the limit?
- **Timing:** Is the 'critical path' short enough to meet our target clock frequency? This is often the most important metric for success.
- **Power:** An estimate of how much power the design will consume.



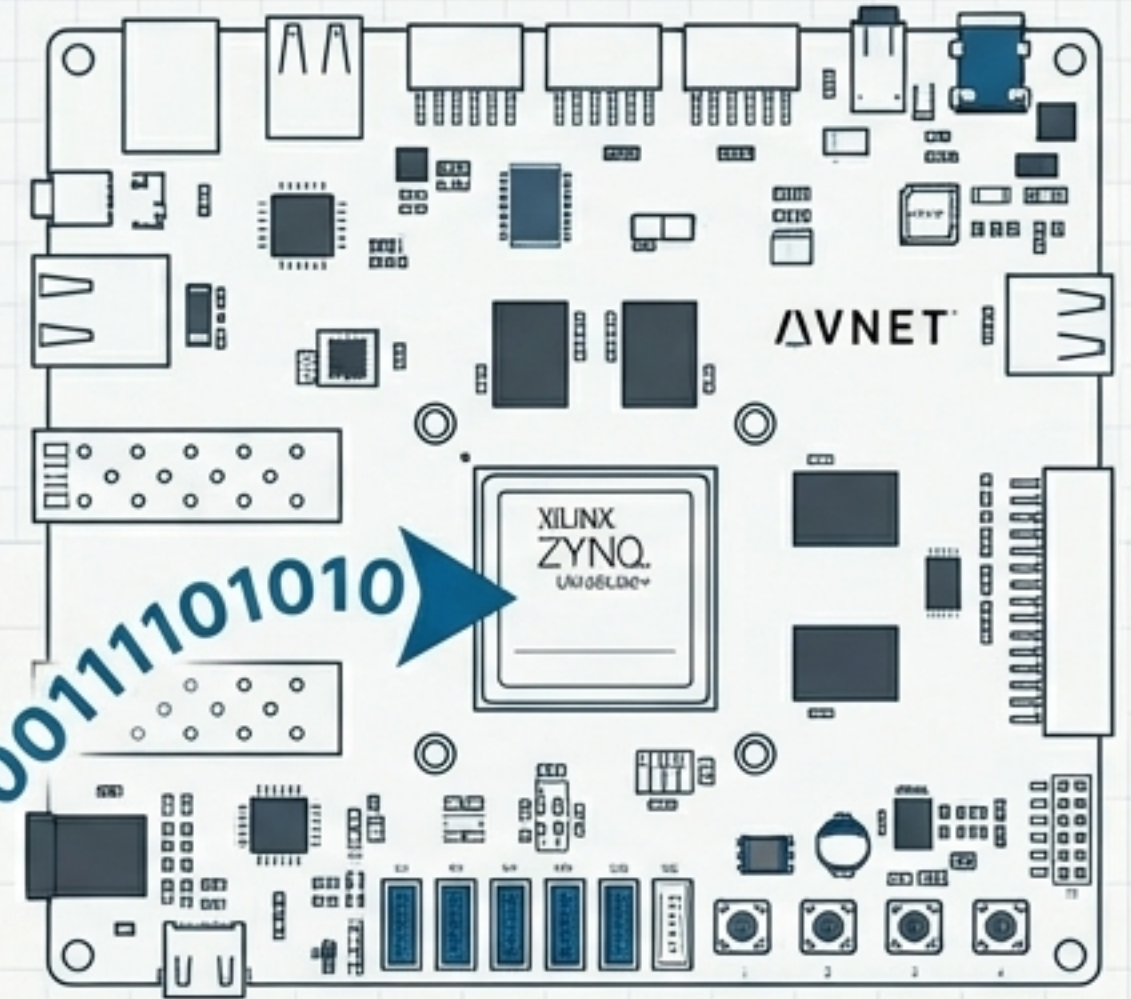
Stage 5: Generating the Final Bitstream

The Final Artifact

The bitstream is a single binary file (.bit) that contains all the configuration data needed to program the FPGA. It effectively acts as the “executable” for the hardware.



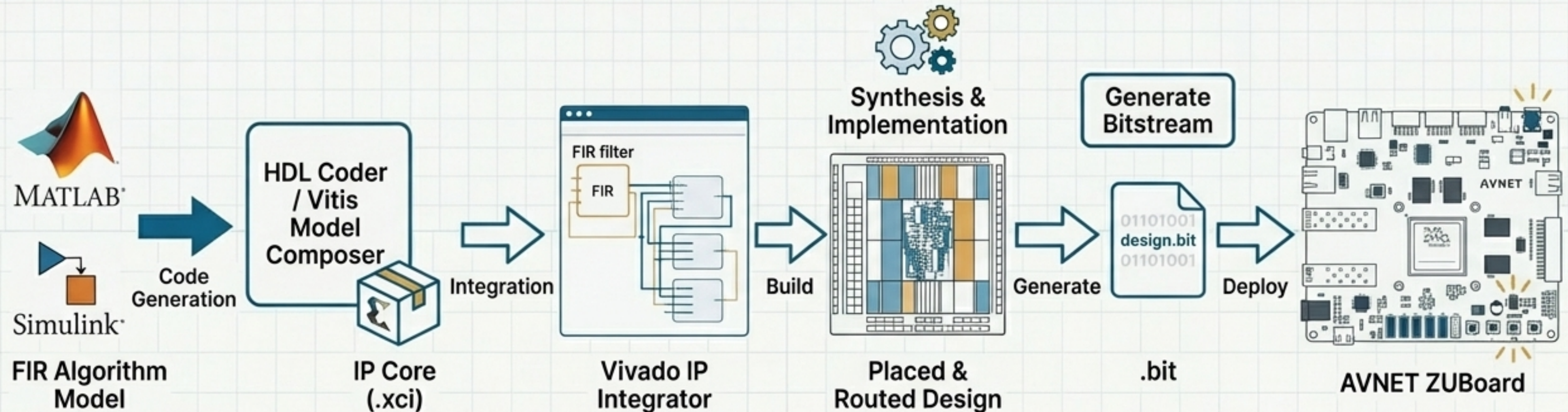
0010111010010011101010



What it Contains

This file holds the state for every configurable logic block (CLB), every switch in the routing fabric, and the configuration for every I/O pin. Loading this bitstream transforms the generic chip into our specific, custom-designed FIR filter system.

The Complete Journey: From High-Level Concept to Hardware Reality



This structured workflow—from high-level modeling and verification to standardized IP integration and automated implementation—empowers developers to confidently tackle complex designs and transform algorithms into high-performance hardware.